

Quantum Real Estate Analyzer 2.0 — Complete System Documentation

Platform: Google Sheets + Google Apps Script

Version: 2.0 (Production)

Source directory: `src/` (17 `.gs` modules + HTML interfaces)

Purpose: End-to-end real estate deal pipeline — from raw lead ingestion through multi-strategy analysis, scoring, offer generation, buyer matching, and CRM sync.

Table of Contents

1. [System Overview](#)
2. [Architecture](#)
3. [Workbook Structure — 27 Sheets](#)
4. [Module Reference](#)
 - [4.1 Config.gs](#)
 - [4.2 Code.gs](#)
 - [4.3 AutomationCenter.gs](#)
 - [4.4 SheetManager.gs](#)
 - [4.5 Logger.gs](#)
 - [4.6 Ingestion.gs](#)
 - [4.7 Dedup.gs](#)
 - [4.8 MarketDataProvider.gs](#)
 - [4.9 MarketIntel.gs](#)
 - [4.10 RepairEngine.gs](#)
 - [4.11 StrategyEngines.gs](#)
 - [4.12 VerdictEngine.gs](#)
 - [4.13 OfferEngine.gs](#)
 - [4.14 MessagingAI.gs](#)
 - [4.15 BuyerMatch.gs](#)
 - [4.16 SpeedToLead.gs](#)
 - [4.17 CRMIntegrations.gs](#)
5. [Full Pipeline Execution Order](#)
6. [Sheet Schemas — Complete Column Definitions](#)
7. [Configuration Reference](#)
8. [Scoring & Decision Logic](#)

- 9. CRM Integrations
 - 10. Automation & Scheduling
 - 11. HTML Interfaces
 - 12. Data Identifiers & Deduplication
 - 13. Key Algorithms
 - 14. Settings Sheet Reference
-

1. System Overview

The Quantum Real Estate Analyzer is a Google Sheets–based investment operating system. It ingests real estate leads from multiple sources, normalizes and deduplicates them, runs every deal through five investment strategy engines (Flip, STR, MTR, LTR, Creative Finance), scores each deal with a composite Deal Score and Risk Score, generates a Verdict classification, produces structured offers across six offer types, matches deals to a buyer database, enforces speed-to-lead SLA compliance, and optionally syncs hot leads to three CRM platforms (SMS-iT, CompanyHub, OhMyLead).

Lead sources supported:

- Browse AI (scraped web listings)
- PropStream (data export)
- MLS (feed data)
- Web & Ad Leads (inbound forms, Facebook Ads, OhMyLead webhooks)

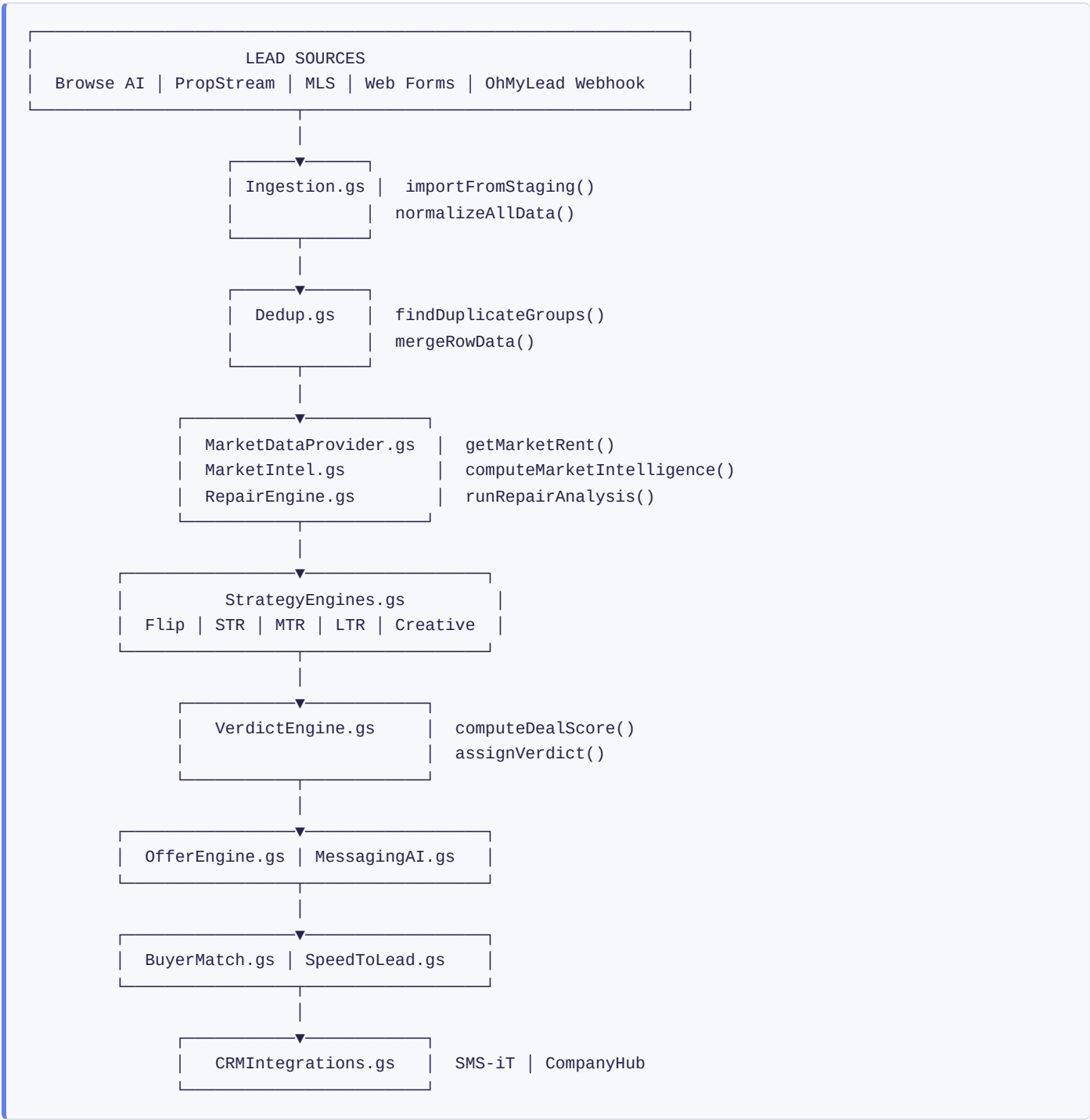
Investment strategies analyzed per deal:

- Fix-and-Flip
- Short-Term Rental (STR / Airbnb)
- Medium-Term Rental (MTR / furnished monthly)
- Long-Term Rental (LTR / traditional)
- Creative Finance (Subject-To, Wrap, Seller Carry, Lease Option, Hybrid)

Outputs per deal:

- Deal Score (0–100), Risk Score (0–100)
 - Verdict: HOT / SOLID / HOLD / PASS
 - Next Action: CALL NOW / MAKE OFFER / WATCH / SKIP / RESEARCH / FOLLOW UP
 - Priority Rank (1 = best deal in the entire pipeline)
 - MAO (Max Allowable Offer) for each strategy
 - Full offer pack (6 offer types with terms)
 - Seller psychology profile + first-touch message
 - Top-3 buyer matches with match scores
 - SLA tier and compliance status
-

2. Architecture



All modules write to and read from the **Master Database** sheet as the central data store. Child sheets (strategy engine sheets, Verdict, Offers, Buyer Matching, Repair Estimator) each hold computed outputs keyed by Deal ID as a foreign key back to Master Database.

3. Workbook Structure — 27 Sheets

Group 1: Import & Staging (5 sheets)

Sheet Name	Purpose
Import Hub	Tracks import record counts, timestamps, status, and action buttons per source
Staging - Browse AI	Raw scraped lead data from Browse AI before normalization
Staging - PropStream	Raw PropStream CSV export data before normalization
Staging - MLS	Raw MLS feed data before normalization
Web & Ad Leads	Inbound leads from website forms, Facebook Ads, and OhMyLead webhooks

Group 2: Core Data (3 sheets)

Sheet Name	Purpose
Master Database	Central normalized deal repository (69 columns, full lifecycle)
Enhanced Deal Analyzer	Deep-dive per-deal workspace mirroring Master DB analysis columns
Lead Scoring & Risk	Breakout of individual scoring components per deal (14 columns)

Group 3: Strategy Engines (5 sheets)

Sheet Name	Purpose
Flip Engine	Fix-and-flip profitability: ARV, rehab, holding costs, ROI (21 columns)
STR Engine	Short-Term Rental cash flow: ADR, occupancy, seasonality, regulation risk (17 columns)
MTR Engine	Medium-Term Rental: furnished monthly rent, vacancy smoothing, stability (16 columns)
LTR Engine	Long-Term Rental: NOI, DSCR, CapEx reserves, rent growth (20 columns)
Creative Finance Engine	Sub2, Wrap, Seller Carry, Lease Option, Hybrid analysis (37 columns)

Group 4: Outputs (6 sheets)

Sheet Name	Purpose
Verdict	Ranked deal leaderboard sorted by Deal Score (19 columns)
Offer & Disposition	Generated offers with terms, response tracking, buyer assignment (19 columns)
Repair Estimator	Line-item rehab cost breakdown by building system (25 columns)
Buyer Database	Buyer profiles with strategy preferences, budgets, qualifications (17 columns)
Buyer Matching Engine	Top-3 buyer matches per deal with scores and dispo status (16 columns)

Sheet Name	Purpose
Post-Sale Tracker	Projected vs. actual performance for closed deals (19 columns)

Group 5: Admin & Config (3 sheets)

Sheet Name	Purpose
Settings	All configurable system parameters (50+ rows, 5 columns)
Dashboard	KPI summary metrics with trends
Control Center	Automation action buttons with run status

Group 6: Logs (3 sheets)

Sheet Name	Purpose
System Log	Timestamped operational events from all modules (4 columns)
Error Log	Errors with module, message, stack trace, resolved status (5 columns)
Sync Log	CRM sync events with system, action, record ID, status (6 columns)

4. Module Reference

4.1 Config.gs

Purpose: Single source of truth for all system constants, thresholds, sheet names, column definitions, and strategy defaults. Also provides helper functions for reading/writing settings from the Settings sheet.

Key exports:

`CONFIG` — Master configuration object containing:

- `CONFIG.SHEETS` — All 27 sheet name strings (e.g., `CONFIG.SHEETS.MASTER_DB = 'Master Database'`)
- `CONFIG.VELOCITY` — DOM-based sales velocity tiers: FAST (≤ 14 DOM, score 90), MOD (≤ 45 , score 70), SLOW (≤ 90 , score 40), STALE (≤ 999 , score 20)
- `CONFIG.EXIT_RISK` — Exit risk tiers with MAO multipliers: LOW (≤ 30 , $\times 1.0$), MOD (≤ 50 , $\times 0.95$), HIGH (≤ 70 , $\times 0.90$), CRIT (≤ 100 , $\times 0.85$)
- `CONFIG.SOM` — Market saturation tiers with verdict boosts: LOW (≤ 30 , +5), MOD (≤ 50 , 0), HIGH (≤ 70 , -5), SATURATED (≤ 100 , -15)
- `CONFIG.SPEED_TO_LEAD` — SLA tiers in minutes: TIER_1 (≤ 5 min, OPTIMAL, no penalty), TIER_2 (≤ 15 , ACCEPTABLE, -5), TIER_3 (≤ 60 , SLOW, -15, escalate), BREACH (> 60 , -25, escalate)
- `CONFIG.REPAIR` — Repair complexity tiers with \$/sqft multiplier ranges: COSMETIC (\$5–\$15/sqft, risk 10), MODERATE (\$15–\$35, risk 30), HEAVY (\$35–\$60, risk 50), FULL_GUT (\$60–\$100, risk 75), TEARDOWN (\$100–

\$150, risk 90)

- `CONFIG.STRATEGIES` — All five strategy engine default values (see Section 7)
- `CONFIG.VERDICT` — Score thresholds: HOT ≥80, SOLID ≥60, HOLD ≥40, PASS <40
- `CONFIG.CRM` — CRM connection configuration (enabled flags, API URLs, keys)
- `CONFIG.AI` — OpenAI settings (enabled, API key, model, max tokens)
- `CONFIG.THEME` — UI color palette
- `CONFIG.COLUMNS` — Complete column header arrays for every sheet

`CacheManager` — Wrapper around `CacheService.getScriptCache()` for short-lived (5-minute default) caching of frequently read data (settings, buyers, ZIP velocity).

Helper functions:

Function	Purpose
<code>getSetting(name, default)</code>	Reads a named setting from the Settings sheet
<code>setSetting(name, value)</code>	Writes a setting to the Settings sheet with timestamp
<code>getVelocityTier(dom)</code>	Returns velocity tier object for a given Days-on-Market value
<code>getExitRiskTier(score)</code>	Returns exit risk tier for a given risk score
<code>getSOMTier(score)</code>	Returns SOM tier for a given saturation score
<code>getSLATier(minutes)</code>	Returns SLA tier for elapsed minutes since lead arrival
<code>getVerdict(score)</code>	Returns verdict object (HOT/SOLID/HOLD/PASS) for a deal score
<code>getRepairTier(complexity)</code>	Returns repair tier config for a complexity level string

4.2 Code.gs

Purpose: Main entry point. Creates the custom Google Sheets menu, launches HTML dialog interfaces, orchestrates the full pipeline, manages time-based triggers, and provides system utility functions.

Key functions:

Function	Purpose
<code>onOpen(e)</code>	Fires on spreadsheet open; builds custom menu, runs health check
<code>createCustomMenu()</code>	Builds the "Quantum Analyzer" menu with all sub-menus
<code>runFullPipeline()</code>	Orchestrates all 10 pipeline steps in sequence with progress toasts
<code>runIngestAndClean()</code>	Calls <code>importFromStaging</code> → <code>normalizeAllData</code> → <code>stampLeadArrival</code>
<code>runAnalyzeAndScore()</code>	Calls <code>computeAllScores</code> → <code>runAllStrategyEngines</code> → <code>runMultiExitComparison</code>
<code>createTriggers()</code>	Deletes old triggers, creates nightly/hourly/5-min time-based triggers

Function	Purpose
<code>nightlyRefresh()</code>	Scheduled: <code>importFromStaging</code> → <code>runAnalyzeAndScore</code> → <code>refreshDashboard</code> → <code>archiveOldLogs</code>
<code>runHealthCheck()</code>	Checks all 27 sheets exist and triggers are configured; returns status object
<code>runHealthCheckWithReport()</code>	UI-facing health check with alert dialog
<code>clearAllLogs()</code>	Clears System Log, Error Log, Sync Log content (keeps headers)
<code>exportSystemConfig()</code>	Writes JSON dump of CONFIG to a temporary "Config Export" sheet
<code>resetToDefaults()</code>	Prompts confirmation, then re-runs <code>initializeSettingsSheet()</code>
<code>openSetupWizard()</code>	Opens setup-wizard HTML dialog (900×700)
<code>openControlCenter()</code>	Opens control-center HTML dialog (1200×800)
<code>openDealAnalyzer()</code>	Opens deal-analyzer HTML dialog (1100×750)
<code>openOfferGenerator()</code>	Opens offer-generator HTML dialog (1000×700)
<code>openBuyerMatcher()</code>	Opens buyer-matcher HTML dialog (1000×700)
<code>openHelpSOP()</code>	Opens help-sop HTML dialog (900×650)

Full pipeline step sequence (`runFullPipeline()`):

1. `runIngestAndClean()` — import, normalize, timestamp leads
2. `runDeduplication()` — find and merge duplicates
3. `runEnrichment()` — market intel, repair analysis, comp confidence
4. `computeAllScores()` — lead scoring and risk scoring
5. `runAllStrategyEngines()` — all 5 strategy engines
6. `generateVerdictRankings()` — deal scoring and ranking
7. `generateOfferPack()` + `generateSellerMessages()` — offers and messaging
8. `runBuyerMatching()` — buyer-to-deal matching
9. `syncToCRMIfEnabled()` — push to connected CRMs
10. `refreshDashboard()` — KPI dashboard update

4.3 AutomationCenter.gs

Purpose: Manages scheduled automation triggers, dashboard KPI aggregation, data enrichment orchestration, Control Center data for the HTML UI, and the Post-Sale feedback loop.

Key functions:

Function	Purpose
<code>runFullPipelineSafe()</code>	Runs <code>runFullPipeline()</code> inside a <code>LockService</code> script lock to prevent concurrent runs

Function	Purpose
<code>setupAutomationTriggers()</code>	Deletes all existing triggers; recreates based on Settings toggles
<code>listTriggers()</code>	Returns array of all active triggers with handler name and event type
<code>removeAllTriggers()</code>	Deletes all project triggers
<code>refreshDashboard()</code>	Aggregates all KPI metrics and writes them to the Dashboard sheet
<code>gatherDashboardMetrics()</code>	Calls all stats functions and assembles the metrics object
<code>getVerdictCounts()</code>	Counts HOT/SOLID/HOLD/PASS records in Master DB
<code>getScoreStatistics()</code>	Calculates average Deal Score and Risk Score
<code>getOfferStatistics()</code>	Counts offers sent and accepted from Offer & Disposition sheet
<code>getLastPipelineRun()</code>	Reads System Log to find the last PIPELINE completed entry
<code>getSystemHealthStatus()</code>	Returns 'HEALTHY' or 'ISSUES' based on <code>runHealthCheck()</code>
<code>runEnrichment()</code>	Calls <code>computeMarketIntelligence</code> + <code>runRepairAnalysis</code> + <code>updateCompConfidence</code>
<code>getControlCenterData()</code>	Returns metrics, triggers, CRM status, log stats, SLA config, recent logs/errors for HTML UI
<code>toggleAutomation(key, enabled)</code>	Updates a setting and recreates triggers
<code>recordPostSaleOutcome(dealId, outcomes)</code>	Writes actual vs. projected metrics to Post-Sale Tracker; updates Status Stage to 'Closed'
<code>generateTuneRecommendations(pv, pfv)</code>	Generates text advice based on price and profit variance percentages
<code>getPostSaleAnalytics()</code>	Aggregates total closed deals, total profit, average variances

Dashboard KPI rows written: Total Leads, HOT Deals, SOLID Deals, HOLD Deals, PASS Deals, Avg Deal Score, Avg Risk Score, STL - Optimal, STL - Breach, Avg Response Time, CRM Synced, CRM Pending, Active Buyers, Offers Sent, Offers Accepted, Last Pipeline Run, System Health.

4.4 SheetManager.gs

Purpose: Creates and initializes all 27 sheets with correct headers, applies formatting, sets conditional formatting rules, applies data validations, and initializes the Settings sheet with default values.

Key functions:

Function	Purpose
<code>initializeAllSheets()</code>	Creates all 27 sheets in correct groups; applies headers, formatting, validations
<code>createStagingSheets()</code>	Creates Import Hub + 3 Staging sheets + Web & Ad Leads
<code>createCoreSheets()</code>	Creates Master Database, Enhanced Deal Analyzer, Lead Scoring & Risk

Function	Purpose
<code>createStrategyEngineSheets()</code>	Creates all 5 strategy engine sheets
<code>createOutputSheets()</code>	Creates Verdict, Offer & Disposition, Repair Estimator, Buyer Database, Buyer Matching, Post-Sale Tracker
<code>createAdminSheets()</code>	Creates Settings, Dashboard, Control Center
<code>createLogSheets()</code>	Creates System Log, Error Log, Sync Log
<code>createSheetWithHeaders(name, headers)</code>	Generic: creates or clears a sheet, writes header row, freezes row 1
<code>applyAllFormatting()</code>	Applies formatting to all sheets by type
<code>formatOperationalSheet(sheet)</code>	Alternating row colors, bold header, freeze pane
<code>formatStrategySheet(sheet)</code>	Strategy-specific formatting with color-coded verdict column
<code>applyVerdictConditionalFormatting(sheet)</code>	Green/Blue/Orange/Red background rules for HOT/SOLID/HOLD/PASS cells
<code>applyScoreConditionalFormatting(sheet)</code>	Color gradient for score columns (0–100)
<code>applyDataValidations()</code>	Applies dropdown validations across all relevant sheets
<code>applyMasterDBValidations()</code>	Property Type, Seller Type, Status Stage, Verdict, SLA Status dropdowns
<code>applyBuyerDBValidations()</code>	Strategy Preference, Risk Tolerance, Active dropdowns
<code>initializeSettingsSheet()</code>	Writes all 50+ default settings rows to the Settings sheet
<code>getColumnByHeader(sheet, header)</code>	Returns 1-indexed column number for a given header string
<code>getColumnMap(sheet)</code>	Returns object mapping header names to 1-indexed column numbers

4.5 Logger.gs

Purpose: Centralized logging for all system events, errors, and CRM sync operations. Writes to three log sheets (System Log, Error Log, Sync Log). Provides diagnostics and log management utilities.

Key functions:

Function	Purpose
<code>logEvent(category, message, details?)</code>	Appends a row to System Log with timestamp, category, message, optional details
<code>logError(module, message, stack?)</code>	Appends to Error Log with timestamp, module, message, stack trace, Resolved='No'
<code>resolveError(rowNum)</code>	Marks a specific Error Log row as Resolved='Yes'
<code>logSync(crmSystem, action, recordId, status, details)</code>	Appends to Sync Log
<code>getRecentLogs(limit)</code>	Returns last N rows from System Log as array of objects

Function	Purpose
<code>getRecentErrors(limit, unresolvedOnly?)</code>	Returns last N error rows, optionally filtering to unresolved only
<code>getRecentSyncLogs(limit)</code>	Returns last N sync log rows
<code>generateDiagnostics()</code>	Runs health check and log stats; returns structured diagnostics object
<code>exportDiagnostics()</code>	Writes diagnostics JSON to a 'Diagnostics' sheet
<code>clearOldLogs(daysToKeep?)</code>	Deletes log rows older than N days (default 30)
<code>getLogStats()</code>	Returns total/error/warning counts and last activity timestamp

4.6 Ingestion.gs

Purpose: Imports raw lead data from all four staging sheet types into the normalized Master Database. Handles field mapping, data normalization (addresses, states, ZIPs, prices), lead ID generation, and arrival timestamp stamping.

Key functions:

Function	Purpose
<code>importFromStaging()</code>	Iterates all three staging sheets + Web & Ad Leads; calls <code>importFromStagingSheet</code> for each
<code>importFromStagingSheet(stagingSheet)</code>	Reads unprocessed rows, maps to Master DB schema, appends, marks processed
<code>mapStagingToMaster(row, headers, source)</code>	Maps raw staging row fields to Master DB column order
<code>importFromBrowseAI()</code>	UI-callable function targeting Staging - Browse AI specifically
<code>importWebAdLeads()</code>	Imports processed Web & Ad Leads rows into Master DB
<code>mapWebLeadToMaster(row, headers)</code>	Maps Web & Ad lead fields to Master DB columns
<code>normalizeAllData()</code>	Iterates all Master DB rows; calls normalization on address, state, ZIP, price
<code>normalizeAddress(address)</code>	Lowercase, trims extra whitespace, strips special characters
<code>normalizeState(state)</code>	Converts full state names to 2-letter codes; uppercases abbreviations
<code>normalizeZip(zip)</code>	Strips non-digits, zero-pads to 5 digits
<code>normalizePrice(price)</code>	Strips \$, commas; parses K/M suffixes; returns float
<code>normalizeNumber(val)</code>	Generic numeric normalizer
<code>normalizeLotSize(lotSize)</code>	Handles mixed "acres"/"sqft" formats
<code>generateDealId()</code>	Returns <code>'Q' + Date.now().toString(36).toUpperCase() + randomString(4)</code>
<code>stampLeadArrival()</code>	Sets Lead Arrival Timestamp on any Master DB row where it is blank

Function	Purpose
<code>updateImportHub(source, count, status)</code>	Updates the Import Hub sheet row for a given source

Normalization rules:

- Addresses: lowercase, stripped of special characters, used as primary dedup key
- States: mapped from full names (e.g., "Florida" → "FL"); stored uppercase
- ZIPs: numeric only, zero-padded to 5 digits
- Prices: strips "\$" and "," symbols; understands "250K" = 250000, "1.2M" = 1200000

4.7 Dedup.gs

Purpose: Detects and merges duplicate records in the Master Database using address+ZIP normalization as the primary key and Listing URL as the secondary key. Keeps the highest-quality record and merges missing fields from lower-quality duplicates.

Key functions:

Function	Purpose
<code>runDeduplication()</code>	Main entry: finds groups → processes → generates report
<code>findDuplicateGroups()</code>	Groups rows by normalized (address+ZIP) key; secondary pass by normalized URL
<code>normalizeForDedup(address, zip)</code>	Lowercases, strips all non-alphanumeric, concatenates with pipe separator
<code>processDuplicates(groups, sheet, headers)</code>	Keeps best row per group; deletes lower-quality duplicates
<code>calculateDataQualityScore(row, headers)</code>	Scores a row 0–100 based on number of non-empty important fields
<code>mergeRowData(best, duplicate, headers)</code>	Copies non-null field values from duplicate into best row where best is empty
<code>areAddressesSimilar(a, b)</code>	Fuzzy match using Levenshtein distance ≤10% of max length
<code>levenshteinDistance(a, b)</code>	Classic dynamic-programming edit distance algorithm
<code>generateDuplicateReport()</code>	Returns stats: total groups, total duplicates, records removed
<code>exportDuplicateReport()</code>	Writes dedup stats to a 'Dedup Report' sheet
<code>deduplicateByURL()</code>	Secondary dedup pass specifically using normalized Listing URL
<code>deduplicateRecentImports()</code>	Runs dedup only on rows imported in the last 24 hours

Dedup key format: `normalizedAddress|normalizedZip`

Data quality scoring: Counts non-empty values across fields: ARV, Asking Price, Beds, Baths, Sqft, Year Built, Seller Type, Motivation Signals, DOM, and others.

4.8 MarketDataProvider.gs

Purpose: Fetches or estimates market data (rental rates, property taxes, ADR) needed by strategy engines. Tries a configured external API first; falls back to internal estimates based on property characteristics and regional factors.

Key functions:

Function	Purpose
<code>getMarketRent(zip, beds, baths, sqft, propType)</code>	Returns estimated fair market rent; tries API then internal estimate
<code>getPropertyTaxes(zip, assessedValue, state)</code>	Returns estimated annual property taxes
<code>getADR(zip, beds, propType)</code>	Returns estimated Average Daily Rate for STR analysis
<code>getMarketDataSummary(zip)</code>	Returns a summary object: medianPrice, priceGrowth, avgDOM, inventory
<code>fetchMarketRentFromAPI(...)</code>	Makes external API call; respects <code>market_data_api_url</code> and <code>market_data_api_key</code> settings
<code>fetchPropertyTaxesFromAPI(...)</code>	External API call for tax data
<code>fetchADRFromAPI(...)</code>	External API call for STR ADR data
<code>estimateMarketRentInternal(...)</code>	Internal estimate: base rent per sqft by bed count, adjusted for ZIP and property type
<code>estimatePropertyTaxesInternal(...)</code>	Internal estimate: 1–2% of assessed value, varying by state
<code>estimateADRInternal(...)</code>	Internal estimate: base ADR by bed count (\$75–\$250+), adjusted for ZIP popularity
<code>getMedianPriceEstimate(zip)</code>	Estimates median home price from ZIP prefix patterns
<code>getPriceGrowthEstimate(zip)</code>	Returns annual price growth estimate (3–7%) by region
<code>getDOMEstimate(zip)</code>	Returns estimated days-on-market by region
<code>getInventoryEstimate(zip)</code>	Returns estimated months of inventory by region
<code>getMarketDataConfig()</code>	Returns whether external API is configured

Fallback logic: If no external API is configured or the call fails, all four functions return internally estimated values. Estimates are based on ZIP prefix (first 3 digits) to approximate regional markets.

4.9 MarketIntel.gs

Purpose: Computes market intelligence fields for every deal in the Master Database: Sales Velocity Score, Exit Speed Tier, Exit Risk Tier, SOM Score, Market Heat Score. These are prerequisite inputs for the strategy engines and VerdictEngine.

Key functions:

Function	Purpose
<code>computeMarketIntelligence()</code>	Iterates Master DB; calculates and writes all market intel columns per row
<code>calculateSalesVelocity(dom, zip)</code>	Base velocity score from DOM tier + ZIP-specific velocity bonus
<code>getZIPVelocityBonus(zip)</code>	Returns ±0 to ±15 adjustment based on ZIP prefix region patterns
<code>computeDOM(row, headers)</code>	Returns DOM value; handles parsing of various date/number formats
<code>calculateExitRisk(deal)</code>	Weighted risk: DOM weight (40%), repair risk (30%), market saturation (20%), comp confidence (10%)
<code>calculateSOMScore(zip, propType)</code>	Estimates market saturation 0–100 based on ZIP and property type inventory levels
<code>getZIPSaturation(zip)</code>	Returns base saturation score 20–65 for a ZIP by prefix region
<code>applySOMImpact(dealScore, somScore)</code>	Applies SOM tier verdict boost/penalty to a score
<code>calculateMarketHeat(velocityScore, exitRisk, somScore)</code>	Weighted composite: velocity 40%, inverse exit risk 35%, inverse SOM 25%
<code>getMarketHeatDescription(score)</code>	Returns human-readable label for heat score
<code>calculateCompConfidence(deal)</code>	Confidence score 0–100 based on presence of ARV, Zestimate, DOM, beds/baths/sqft
<code>updateAllMarketData()</code>	Re-runs market intel computation for all rows
<code>calculateAbsorptionRate(dom, inventory)</code>	Returns months of supply; used for SOM interpretation
<code>getInventoryInterpretation(months)</code>	Returns buyer/seller market characterization

Exit Risk calculation weights:

- DOM component (40%): $\text{normalized DOM} / 180 \times 100$, capped at 100
- Repair risk (30%): from Repair Risk Score column
- Market saturation (20%): from SOM Score
- Comp confidence (10%): inverse of Comp Confidence Score

4.10 RepairEngine.gs

Purpose: Estimates rehabilitation costs for each deal by property size, age, type, and condition notes. Outputs repair complexity tier, low/high cost estimates, and a repair risk score. Also maintains the Repair Estimator sheet with line-item breakdowns.

Key functions:

Function	Purpose
<code>runRepairAnalysis()</code>	Iterates Master DB; calls <code>analyzeRepairs</code> per deal; writes to Repair Estimator sheet and back to Master DB
<code>analyzeRepairs(deal)</code>	Main analysis: determines complexity tier, estimates costs, computes risk score
<code>determineComplexityTier(deal)</code>	Returns COSMETIC/MODERATE/HEAVY/FULL_GUT/TEARDOWN based on year built, sqft, condition notes
<code>calculateRepairRisk(complexityTier, yearBuilt, sqft)</code>	Returns risk score from CONFIG.REPAIR tier + age penalty
<code>estimateRepairCategories(deal, complexityTier)</code>	Returns line-item estimates for Roof, HVAC, Plumbing, Electrical, Foundation, Kitchen, Bathrooms, Flooring, Paint, Windows/Doors, Exterior, Landscaping, Other
<code>extractConditionNotes(deal)</code>	Reads Motivation Signals and Description fields for condition keywords
<code>getRepairNotes(complexityTier)</code>	Returns human-readable notes per complexity tier
<code>getRepairEstimate(dealId)</code>	Retrieves saved estimate from Repair Estimator sheet for a given Deal ID
<code>updateRepairEstimate(dealId, updates)</code>	Updates specific fields in the Repair Estimator sheet for a deal
<code>quickRepairCalc(sqft, complexityTier)</code>	Returns quick low/high estimate without writing to any sheet

Complexity tier determination rules:

- Year built < 1950 → escalates to HEAVY minimum
- Year built 1950–1979 → escalates to MODERATE minimum
- Sqft > 3000 → adds complexity tier
- Keywords in condition notes: "needs work", "as-is", "investor special", "gut", "teardown" each increase tier

Cost multipliers (\$/sqft):

- COSMETIC: \$5–\$15
- MODERATE: \$15–\$35
- HEAVY: \$35–\$60
- FULL_GUT: \$60–\$100
- TEARDOWN: \$100–\$150

4.11 StrategyEngines.gs

Purpose: The largest module (~1,462 lines). Runs all five investment strategy analyses for every deal in the Master Database. Computes strategy-specific scores (0–100), MAO values, cash flows, returns, and verdicts. Also runs multi-exit comparison to identify the best strategy per deal.

Flip Engine

Key functions: `runFlipEngine()`, `analyzeFlipDeal(deal)`, `runFlipEngineOnly()`

Calculation logic:

```
Rehab Low = sqft × CONFIG.REPAIR[tier].lowMultiplier
Rehab High = sqft × CONFIG.REPAIR[tier].highMultiplier
Rehab Mid = (Rehab Low + Rehab High) / 2

Holding Cost = Purchase Price × 0.01 × holdingMonths
Agent Fees = ARV × 0.06
Closing Costs = ARV × 0.03

Total Costs = Purchase Price + Rehab Mid + Holding Cost + Agent Fees + Closing Costs

Profit if Rehab Mid = ARV - Total Costs
ROI = Profit / (Purchase Price + Rehab Mid) × 100

MAO Flip = ARV × 0.70 - Rehab Mid (70% rule)
```

Flip Score factors: Profit margin, ROI, market velocity, exit risk.

STR Engine

Key functions: `runSTREngine()`, `analyzeSTRDeal(deal)`, `runSTREngineOnly()`

Calculation logic:

```
STR Monthly Gross = ADR × 30 × occupancy × seasonalityIndex × (1 - regulationRisk × 0.3)
STR Monthly Net = Gross - (cleaningCost × turns/month) - (Gross × mgmtFee) - (Gross × platformFee)
STR Annual Net = Monthly Net × 12

Break-Even Occupancy = Monthly Expenses / (ADR × 30)
Cash-on-Cash = (Annual Net / (Purchase Price + Rehab + Furnishing)) × 100

MAO STR = (Annual Net / targetCapRate) - Rehab - Furnishing
```

Defaults: occupancy 65%, management fee 20%, cleaning/turn \$75, furnishing \$8,000 + \$2,000 setup, platform fee 3%, regulation risk 0.5.

MTR Engine

Key functions: `runMTREngine()`, `analyzeMTRDeal(deal)`, `runMTREngineOnly()`

Calculation logic:

```

Turns Per Year = 12 / avgStayLength (default 3 months)
Vacancy Smoothing Score = based on stay length and gap between stays
Furniture Amortization = furnitureCost / 24 per month

MTR Monthly Gross = furnishedMonthlyRent × (1 - vacancyRate)
MTR Monthly Net = Gross - utilitiesBundle - furnitureAmort - (Gross × mgmtFee)
MTR Annual Net = Monthly Net × 12

MTR Stability Score = weighted average of vacancy smoothing + demand score
MTR Advantage Index = MTR Net vs LTR Net comparison

MAO MTR = (Annual Net / targetCapRate) - Rehab - Furnishing

```

Defaults: avg stay 3 months, utilities bundle \$200/month, furniture amortized over 24 months, management fee 12%.

LTR Engine

Key functions: `runLTREngine()`, `analyzeLTRDeal(deal)`, `runLTREngineOnly()`

Calculation logic:

```

Effective Gross Income = MarketRent × (1 - vacancyRate)
Total OpEx = Maintenance + Taxes + Insurance + CapEx + PM Fee
NOI Monthly = EGI - Total OpEx
NOI Annual = NOI Monthly × 12

DSCR = NOI Annual / Annual Mortgage Payment (target ≥ 1.25)

LTR Cash-on-Cash = Annual Net Cash Flow / Down Payment × 100
Hold Quality Score = DSCR score + rent growth + market heat composite

MAO LTR = (NOI Annual / targetCapRate)

```

Defaults: vacancy 8%, maintenance reserve 10%, CapEx reserve 5%, property management 10%, rent growth 3% annually, target DSCR 1.25.

Creative Finance Engine

Key functions: `runCreativeEngine()`, `analyzeCreativeDeal(deal)`, `runCreativeEngineOnly()`

Analyzes five creative structures per deal:

Sub-To (Subject-To existing mortgage):

- Entry Cost = Down payment (5% of asking)
- Monthly Cash Flow = Market Rent – Existing Monthly Payment – operating expenses
- Equity Position = ARV – Existing Mortgage Balance
- Viable if: equity > 20% and cash flow > 0

Wrap Mortgage:

- Buyer pays investor at higher rate; investor continues paying seller's mortgage
- Wrap Spread = buyer rate – existing rate (min 1%, max 3%)

- Monthly Cash Flow = (Wrap Payment – Existing Payment) × loan balance
- Viable if spread ≥ 1% and down payment collected

Seller Carry:

- Seller finances purchase at negotiated rate (default 6%, 5-year term, 3-year balloon)
- Monthly Payment calculated via standard mortgage amortization
- Viable if seller has sufficient equity

Lease Option:

- Option Fee = 3% of purchase price
- Monthly Rent = market rent
- Rent Credit = 25% of each payment toward purchase
- Strike Price = ARV at time of option exercise
- Viable if monthly cash flow positive

Hybrid:

- Combines two structures (e.g., Sub2 + Lease Option)
- Viable if at least two individual structures are viable

getMortgageDataWithAssumptions_: Reads Existing Mortgage Balance with ASSUMED/KNOWN flag from Master DB. Uses assumption-based defaults when ASSUMED.

Multi-Exit Comparison

`runMultiExitComparison()` — Runs all five engines on all deals, then compares strategy scores to select the single best strategy per deal. Writes `Best Strategy`, `Strategy Rationale`, and `Multi-Exit Summary` (a side-by-side text table of all five scores) back to Master DB.

MAO Final = maximum viable MAO across all five strategies.

4.12 VerdictEngine.gs

Purpose: Computes the composite Deal Score (0–100) and Risk Score (0–100) for every deal in the Master Database. Assigns Verdict, Next Action, and Priority Rank. Updates the Verdict sheet leaderboard and Lead Scoring & Risk sheet.

Key functions:

Function	Purpose
<code>generateVerdictRankings()</code>	Main entry: computes all scores → assigns verdicts → writes Verdict sheet sorted by rank
<code>computeDealScore(deal)</code>	Weighted composite of 8 sub-scores
<code>computeProfitScore(deal)</code>	Score based on best MAO vs asking price margin
<code>computeMarketConditionScore(deal)</code>	Score from velocity + exit risk + market heat

Function	Purpose
<code>computeMotivationScore(deal)</code>	Score from seller type and motivation signal keywords
<code>computePropertyQualityScore(deal)</code>	Score from year built, property type, comp confidence
<code>computeSTLScore(deal)</code>	Score based on SLA tier (OPTIMAL=100, BREACH=0)
<code>computeSOMImpact(deal)</code>	Returns SOM verdict boost/penalty from CONFIG.SOM tiers
<code>computeStrategyBonus(deal)</code>	+5 for HOT verdict deals with high strategy score
<code>computeRiskScore(deal)</code>	Weighted risk: repair risk 35%, exit risk 30%, market saturation 20%, comp uncertainty 15%
<code>assignVerdict(score)</code>	Returns HOT/SOLID/HOLD/PASS based on CONFIG.VERDICT thresholds
<code>determineNextAction(deal, verdict)</code>	Maps verdict + urgency signals to: CALL NOW / MAKE OFFER / WATCH / SKIP / RESEARCH / FOLLOW UP
<code>computeAllScores()</code>	Batch-runs computeDealScore + computeRiskScore for all Master DB rows
<code>updateLeadScoringSheet()</code>	Writes individual sub-score components to Lead Scoring & Risk sheet
<code>getVerdictForDeal(dealId)</code>	Returns verdict data for a specific deal from Master DB
<code>getTopDeals(n, minVerdict)</code>	Returns top N deals by Deal Score, optionally filtered by verdict
<code>getDealsByVerdict(verdict)</code>	Returns all deals matching a specific verdict
<code>computeSellerResponseScore(deal)</code>	Returns score based on previous contact attempts vs. response

Deal Score weights:

- Profit Score: 30%
- Market Conditions: 25%
- Motivation: 15%
- Property Quality: 10%
- Speed-to-Lead: 10%
- SOM Impact: applied as additive boost/penalty
- Strategy Bonus: additive

Risk Score weights:

- Repair Risk Score: 35%
- Exit Risk Score: 30%
- Market Saturation (SOM): 20%
- Comp Confidence (inverse): 15%

4.13 OfferEngine.gs

Purpose: Generates structured offer packages for every deal in the Master Database. Produces one recommended offer type plus five alternative offer structures. Writes results to the Offer & Disposition sheet and summarizes back to Master DB.

Key functions:

Function	Purpose
<code>generateOfferPack()</code>	Iterates Master DB; calls <code>generateDealOfferPack</code> per deal; writes to Offer & Disposition sheet
<code>generateDealOfferPack(deal)</code>	Generates all 6 offer types; calls <code>determineBestOffer</code> ; returns complete offer object
<code>generateCashOffer(deal)</code>	Cash offer at MAO Final with 10-day close, inspection waiver terms
<code>generateSub2Offer(deal)</code>	Subject-To terms: entry cost, existing payment assumption, equity position
<code>generateWrapOffer(deal)</code>	Wrap mortgage terms: spread, new payment, down payment requirement
<code>generateSellerCarryOffer(deal)</code>	Seller-carry terms: interest rate, term, balloon payment schedule
<code>calculateBalloonBalance(principal, rate, term, balloon)</code>	Computes remaining balance at balloon date using standard amortization
<code>generateLeaseOptionOffer(deal)</code>	Lease-option terms: option fee, monthly rent, rent credit, strike price, term
<code>generateHybridOffer(deal)</code>	Selects best two viable structures and combines them
<code>determineBestOffer(offerPack, deal)</code>	Selects recommended offer type based on deal characteristics and strategy scores
<code>getOfferPackForDeal(dealId)</code>	Retrieves saved offer pack from Offer & Disposition sheet
<code>updateOfferStatus(dealId, updates)</code>	Updates response, counter terms, status fields after offer is sent

Offer selection logic for `determineBestOffer` :

1. If Creative Finance Score > 70 and seller has equity → favor Sub2 or Carry
2. If Flip Score > 70 and repair risk is low → favor Cash
3. If STR Score > 75 → favor Cash (for STR acquisition)
4. If LTR DSCR ≥ 1.25 → favor Cash or Seller Carry
5. Default → Cash offer

4.14 MessagingAI.gs

Purpose: Generates personalized seller outreach messages, psychological profiles, and follow-up sequence tags for each deal. Uses rule-based generation by default; optionally calls OpenAI API if configured.

Key functions:

Function	Purpose
<code>generateSellerMessages()</code>	Iterates Master DB; generates message, profile, follow-up tag, deniability angle per deal
<code>generateFirstTouchMessage(deal)</code>	Selects tone and generates appropriate first-touch message
<code>determineTone(deal)</code>	Returns 'empathetic', 'professional', or 'investor' based on seller type and motivation signals
<code>generateEmpatheticMessage(deal)</code>	For distressed sellers: acknowledges situation, offers solutions
<code>generateProfessionalMessage(deal)</code>	For agent/listed properties: professional, data-focused tone
<code>generateInvestorMessage(deal)</code>	For investor-held properties: investor-to-investor pitch
<code>generateStandardMessage(deal)</code>	Fallback message when seller type is unknown
<code>determineFollowUpSequence(deal)</code>	Returns follow-up tag: 'immediate', 'standard', 'nurture', 'low-priority'
<code>getFollowUpSequence(tag)</code>	Returns sequence descriptor for the assigned tag
<code>generatePsychologyProfile(deal)</code>	Returns psychological profile string based on seller type + motivation signals
<code>assessDeniabilityAngle(deal)</code>	Returns strategic deniability framing for the outreach approach
<code>generateAIMessage(deal)</code>	If OpenAI enabled, builds prompt and calls API; otherwise calls rule-based generator
<code>buildAIPrompt(deal)</code>	Constructs system + user prompt for OpenAI with deal details
<code>callOpenAI(prompt)</code>	Makes HTTP call to OpenAI API; respects <code>ai_openai_api_key</code> and <code>ai_model</code> settings
<code>getMessageForDeal(dealId)</code>	Returns saved message data for a specific deal
<code>regenerateMessage(dealId)</code>	Regenerates and overwrites message for a specific deal

Tone selection rules:

- Estate/Bank/REO seller → empathetic
- Agent-listed → professional
- Investor-held → investor
- Owner with strong motivation signals → empathetic
- Otherwise → standard

Follow-up tag rules:

- Verdict HOT + CALL NOW → 'immediate'
- Verdict SOLID + MAKE OFFER → 'standard'

- Verdict HOLD → 'nurture'
- Verdict PASS or SLA BREACH → 'low-priority'

4.15 BuyerMatch.gs

Purpose: Matches every deal in the Master Database to the top-3 buyers from the Buyer Database using a weighted scoring model. Writes match results to the Buyer Matching Engine sheet and provides buyer CRUD operations.

Key functions:

Function	Purpose
<code>runBuyerMatching()</code>	Loads all deals and active buyers; calculates matches; writes to Buyer Matching Engine sheet
<code>loadBuyerDatabase(sheet)</code>	Returns array of buyer objects where Active = 'Yes'
<code>matchBuyersToDeal(deal, buyers)</code>	Returns top-5 scored matches sorted descending
<code>calculateMatchScore(deal, buyer)</code>	Returns 0–100 weighted match score
<code>getMatchReasons(deal, buyer)</code>	Returns comma-separated list of matching criteria
<code>suggestDispoAction(deal, matches)</code>	Returns action string based on top match score
<code>addBuyer(buyerData)</code>	Generates Buyer ID ('B' + timestamp base36), appends row to Buyer Database
<code>updateBuyer(buyerId, updates)</code>	Updates specific fields for an existing buyer
<code>getBuyer(buyerId)</code>	Returns buyer object for a given Buyer ID
<code>getActiveBuyers()</code>	Returns all buyers where Active = 'Yes'
<code>getMatchesForDeal(dealId)</code>	Returns saved match row from Buyer Matching Engine sheet
<code>getBuyerStatistics()</code>	Returns total/active counts and strategy preference breakdown
<code>assignDealToBuyer(dealId, buyerId)</code>	Records assignment in Offer & Disposition; increments buyer's deal count

Match score weights:

Criterion	Points	Notes
ZIP match (exact)	40	Full credit for exact 5-digit ZIP match
ZIP match (area)	20	Partial credit for same first 3 digits
Strategy match	25	Full credit if buyer strategy = deal strategy or 'Any'
Strategy related	15	Partial credit (e.g., Wholesale buyer for Flip deal)
Budget (exact range)	20	Asking price within buyer's Min–Max range
Budget (near range)	10	Price within ±20% of buyer's range

Criterion	Points	Notes
Property type	10	Buyer's preferred types include deal property type
Risk tolerance	5	Risk tolerance compatible with deal's Risk Score

Dispo action logic:

- Match score ≥ 80 → "Send to [buyer] immediately"
- Score 60–79 → "Contact [buyer] with details"
- Score 40–59 → "Blast to [N] potential buyers"
- Score < 40 → "Hold for better buyer match"

4.16 SpeedToLead.gs

Purpose: Monitors lead arrival timestamps and enforces SLA compliance. Computes SLA tier and status for every lead, triggers escalations for slow or breached leads, and optionally creates CRM tasks for escalated deals.

Key functions:

Function	Purpose
<code>computeSpeedToLeadScores()</code>	Iterates Master DB; calculates minutes elapsed since Lead Arrival Timestamp; writes SLA Tier and SLA Status
<code>checkSpeedToLeadSLA()</code>	Scheduled every 5 minutes: calls <code>computeSpeedToLeadScores</code> , then <code>processSTLEscalations</code>
<code>processSTLEscalations()</code>	Finds SLOW or BREACH records not yet contacted; calls <code>executeEscalation</code> per record
<code>queueCRMEscalation(deal)</code>	Queues a CRM task creation for a deal needing escalation
<code>processSpeedToLeadQueue()</code>	Processes all pending escalation queue items
<code>runSLAEscalations()</code>	Manual trigger for escalation processing
<code>recordFirstContact(dealId, contactedAt?)</code>	Sets Last Contacted At timestamp; stops SLA clock for a deal
<code>getSTLStatistics()</code>	Returns counts of OPTIMAL/ACCEPTABLE/SLOW/BREACH leads and average response time
<code>getSLAConfig()</code>	Returns SLA tier thresholds from <code>CONFIG.SPEED_TO_LEAD</code>
<code>updateSLAConfig(updates)</code>	Updates SLA threshold settings in Settings sheet
<code>executeEscalation(deal)</code>	Sends notification and creates CRM task; logs escalation
<code>sendSTLNotification(deal)</code>	Internal: sends email or CRM alert for escalated deal
<code>getSTLDashboardData()</code>	Returns SLA summary data for the Control Center HTML UI

SLA tier assignment (from `CONFIG.SPEED_TO_LEAD`):

Tier	Threshold	Status	Score Penalty	Escalation
TIER_1	≤ 5 minutes	OPTIMAL	0	No
TIER_2	≤ 15 minutes	ACCEPTABLE	-5	No
TIER_3	≤ 60 minutes	SLOW	-15	Yes
BREACH	> 60 minutes	BREACH	-25	Yes

SLA clock: Starts at Lead Arrival Timestamp (set on import). Stops when Last Contacted At is recorded via `recordFirstContact()`.

4.17 CRMIntegrations.gs

Purpose: Syncs deal data to external CRM platforms (SMS-iT, CompanyHub) and handles inbound OhMyLead webhooks. All outbound calls use a shared retry helper with exponential backoff. Deploys as a Google Apps Script Web App to receive inbound webhooks.

Key functions:

Function	Purpose
<code>crmFetch(url, options, service, maxRetries)</code>	Shared HTTP helper: up to 2 retries with 1s/2s backoff; logs to Sync Log; returns standardized response object
<code>hasCRMCredentials(crmType)</code>	Returns true if API URL and key are both configured for the given CRM
<code>syncToCRMIfEnabled()</code>	Checks which CRMs are enabled in Settings; calls appropriate sync functions
<code>syncToSMSiT()</code>	Syncs HOT and SOLID leads to SMS-iT CRM via REST API
<code>buildSMSiLeadPayload(row, headers, colMap)</code>	Builds SMS-iT contact JSON payload from deal row
<code>sendToSMSiT(apiUrl, apiKey, leadData)</code>	POST to <code>{apiUrl}/contacts</code> with Bearer token auth
<code>syncToCompanyHub()</code>	Syncs all non-PASS leads to CompanyHub via REST API
<code>buildCompanyHubDealPayload(row, headers, colMap)</code>	Builds CompanyHub deal JSON payload
<code>mapVerdictToStage(verdict)</code>	HOT → Qualified, SOLID → Interested, HOLD → Nurturing, PASS → Disqualified
<code>sendToCompanyHub(apiUrl, apiKey, dealData)</code>	POST to <code>{apiUrl}/deals</code> with Bearer token auth
<code>syncToOhMyLead()</code>	OhMyLead is inbound-only; logs configuration status
<code>doPost(e)</code>	Web App handler: validates JSON body, optionally validates webhook secret, maps fields, calls <code>addWebAdLead()</code>
<code>doGet(e)</code>	Web App handler: returns service status JSON for webhook verification

Function	Purpose
<code>addWebAdLead(leadData)</code>	Appends a new row to Web & Ad Leads sheet with Lead Arrival Timestamp
<code>getCRMStatus()</code>	Returns enabled/configured/hasCredentials status for all three CRMs
<code>testCRMConnection(crmType)</code>	Makes live GET to <code>{apiUrl}/me</code> to verify credentials
<code>getCRMSyncStats()</code>	Counts synced vs. pending records in Master DB
<code>markAsSynced(dealId, crmRecordId)</code>	Updates CRM Synced='Yes' and CRM Record ID for a deal
<code>createCRMEscalationTask(escalation)</code>	Creates a high-priority task in CompanyHub for a breached lead (called by SpeedToLead)

Sync filter rules:

- SMS-iT: only syncs HOT and SOLID leads
- CompanyHub: syncs all non-PASS leads
- Both: skips records already marked CRM Synced = 'Yes'

OhMyLead inbound field mapping: `name/full_name`, `email`, `phone/phone_number`, `property_address/address`, `city`, `state`, `zip/postal_code`, `asking_price/price`, `motivation/reason_for_selling`, `campaign/ad_campaign`, `ad_set/adset`, `notes/comments`

5. Full Pipeline Execution Order

```
runFullPipeline()
├─ 1. runIngestAndClean()
│   ├── importFromStaging() [Ingestion.gs]
│   ├── normalizeAllData() [Ingestion.gs]
│   └─ stampLeadArrival() [Ingestion.gs]
├─ 2. runDeduplication()
│   ├── findDuplicateGroups() [Dedup.gs]
│   ├── processDuplicates() [Dedup.gs]
│   └─ generateDuplicateReport() [Dedup.gs]
├─ 3. runEnrichment()
│   ├── computeMarketIntelligence() [MarketIntel.gs]
│   ├── runRepairAnalysis() [RepairEngine.gs]
│   └─ updateCompConfidence() [AutomationCenter.gs]
├─ 4. computeAllScores()
│   ├── computeDealScore() per row [VerdictEngine.gs]
│   └─ computeRiskScore() per row [VerdictEngine.gs]
├─ 5. runAllStrategyEngines()
│   ├── runFlipEngine() [StrategyEngines.gs]
│   ├── runSTREngine() [StrategyEngines.gs]
│   ├── runMTREngine() [StrategyEngines.gs]
│   ├── runLTREngine() [StrategyEngines.gs]
│   └─ runCreativeEngine() [StrategyEngines.gs]
├─ 6. generateVerdictRankings()
│   ├── assignVerdict() per deal [VerdictEngine.gs]
│   ├── determineNextAction() [VerdictEngine.gs]
│   └─ write Verdict sheet [VerdictEngine.gs]
├─ 7. generateOfferPack()
│   └─ generateDealOfferPack() each [OfferEngine.gs]
├─ 7b. generateSellerMessages()
│   └─ generateFirstTouchMessage() [MessagingAI.gs]
├─ 8. runBuyerMatching()
│   └─ matchBuyersToDeal() each [BuyerMatch.gs]
├─ 9. syncToCRMIfEnabled()
│   ├── syncToSMSiT() [CRMIntegrations.gs]
│   └─ syncToCompanyHub() [CRMIntegrations.gs]
└─ 10. refreshDashboard() [AutomationCenter.gs]
```

6. Sheet Schemas — Complete Column Definitions

Column annotations: **M** = Manual input, **C** = Calculated, **S** = System-generated

Master Database (69 columns)

Identity & Source (cols 1–13)

#	Column	Type	M/C/S
1	Deal ID	string (Q + base36 timestamp + 4-char random)	S
2	Source Platform	string (Browse AI / PropStream / MLS / Web / OhMyLead)	S
3	Listing URL	url	M
4	Address	string (normalized)	M → S
5	City	string	M
6	State	string (2-letter)	M → S
7	ZIP	string (5-digit)	M → S
8	County	string	M
9	Lat	number	M
10	Lng	number	M
11	Imported At	datetime	S
12	Lead Arrival Timestamp	datetime	S
13	Source Campaign	string	M

Property Basics (cols 14–20)

#	Column	Type	M/C/S
14	Asking Price	currency	M
15	Beds	integer	M
16	Baths	number	M
17	Sqft	integer	M
18	Lot Size	string/number	M
19	Year Built	integer	M
20	Property Type	enum: SFR, Duplex, Triplex, Fourplex, Condo, Townhouse, Mobile Home, Multi-Family, Land, Commercial, Other	M

Seller Signals (cols 21–24)

#	Column	Type	M/C/S
21	Seller Type	enum: Owner, Agent, Investor, Bank/REO, Estate, Unknown	M
22	Motivation Signals	string (free text)	M
23	Seller Psychology Profile	string	C (MessagingAI.gs)
24	Contact Quality Score	number (0–100)	C

Market Intelligence (cols 25–30)

#	Column	Type	M/C/S
25	DOM	integer	M
26	Sales Velocity Score	number (0–100)	C (MarketIntel.gs)
27	Exit Speed Tier	enum: FAST, MOD, SLOW, STALE	C (MarketIntel.gs)
28	Exit Risk Tier	enum: LOW, MOD, HIGH, CRIT	C (MarketIntel.gs)
29	SOM Score	number (0–100)	C (MarketIntel.gs)
30	Market Heat Score	number (0–100)	C (MarketIntel.gs)

Costs & Rehab (cols 31–34)

#	Column	Type	M/C/S
31	Repair Complexity Tier	enum: COSMETIC, MODERATE, HEAVY, FULL_GUT, TEARDOWN	C (RepairEngine.gs)
32	Est Rehab Low	currency	C (RepairEngine.gs)
33	Est Rehab High	currency	C (RepairEngine.gs)
34	Repair Risk Score	number (0–100)	C (RepairEngine.gs)

Exit Values (cols 35–37)

#	Column	Type	M/C/S
35	ARV	currency (After Repair Value)	M
36	Zestimate	currency	M
37	Comp Confidence Score	number (0–100)	C

Mortgage Data — Creative Finance inputs (cols 38–41)

#	Column	Type	M/C/S
38	Existing Mortgage Balance	currency	M (ASSUMED or KNOWN)
39	Existing Monthly Payment	currency	M
40	Existing Interest Rate	percentage	M

#	Column	Type	M/C/S
41	Mortgage Data Source	enum: ASSUMED, KNOWN	M

MAO Variants (cols 42–47)

#	Column	Type	M/C/S
42	MAO Flip	currency	C (StrategyEngines.gs)
43	MAO STR	currency	C
44	MAO MTR	currency	C
45	MAO LTR	currency	C
46	MAO Creative	currency	C
47	MAO Final	currency (best of all MAOs)	C

Strategy Outputs (cols 48–50)

#	Column	Type	M/C/S
48	Best Strategy	string: Flip / STR / MTR / LTR / Creative	C (StrategyEngines.gs)
49	Strategy Rationale	string	C
50	Multi-Exit Summary	string (all 5 strategies compared)	C

Offer Engine (cols 51–54)

#	Column	Type	M/C/S
51	Offer Type Recommended	string: Cash / Sub2 / Wrap / Seller Carry / Lease Option / Hybrid	C (OfferEngine.gs)
52	Offer Price Target	currency	C
53	Offer Terms Summary	string	C
54	Offer Risk Notes	string	C

Verdict (cols 55–59)

#	Column	Type	M/C/S
55	Deal Score	number (0–100)	C (VerdictEngine.gs)
56	Risk Score	number (0–100)	C
57	Verdict	enum: HOT (≥80) / SOLID (≥60) / HOLD (≥40) / PASS (<40)	C
58	Next Action	enum: CALL NOW / MAKE OFFER / WATCH / SKIP / RESEARCH / FOLLOW UP	C
59	Priority Rank	integer (1 = best)	C

Messaging (cols 60–62)

#	Column	Type	M/C/S
60	Seller Message	string	C (MessagingAI.gs)
61	Follow-Up Tag	string	C
62	Deniability Angle	string	C

Workflow / CRM (cols 63–69)

#	Column	Type	M/C/S
63	Status Stage	enum: New Lead / Contacted / Analyzing / Offer Sent / Negotiating / Under Contract / Due Diligence / Closed / Dead / On Hold	M
64	Assigned To	string	M
65	CRM Synced	enum: Yes / No	S
66	CRM Record ID	string	S
67	Last Contacted At	datetime	M/S
68	SLA Tier	enum: TIER_1 / TIER_2 / TIER_3 / BREACH	C (SpeedToLead.gs)
69	SLA Status	enum: OPTIMAL / ACCEPTABLE / SLOW / BREACH	C

Verdict Sheet (19 columns — all Calculated)

#	Column	Notes
1	Rank	1 = best deal
2	Deal ID	FK to Master Database
3	Address	Copied
4	City	Copied
5	ZIP	Copied
6	Asking Price	Copied
7	ARV	Copied
8	Deal Score	0–100
9	Risk Score	0–100
10	Verdict	HOT/SOLID/HOLD/PASS with conditional formatting
11	Best Strategy	Winning strategy

#	Column	Notes
12	Offer Type	Recommended offer structure
13	Offer Target	Target offer price
14	Exit Speed Tier	FAST/MOD/SLOW/STALE
15	SOM Score	0–100
16	SLA Status	OPTIMAL/ACCEPTABLE/SLOW/BREACH
17	Next Action	Recommended next step
18	Seller Message Preview	First 100 chars of seller message
19	Action Link	Listing URL

Buyer Database (17 columns)

#	Column	Type	M/C/S
1	Buyer ID	string (B + timestamp base36)	S
2	Buyer Name	string	M
3	Company	string	M
4	Email	string	M
5	Phone	string	M
6	ZIPs	string (comma-separated)	M
7	Strategy Preference	enum: Wholesale / Flip / STR / MTR / LTR / Creative / Any	M
8	Budget Min	currency	M
9	Budget Max	currency	M
10	Min DSCR	number	M
11	Yield Preference	string	M
12	Risk Tolerance	enum: Conservative / Moderate / Aggressive	M
13	Preferred Property Types	string	M
14	Active	enum: Yes / No	M
15	Last Deal Date	date	M/S
16	Total Deals Closed	integer	M/S
17	Notes	string	M

Offer & Disposition (19 columns)

#	Column	Type	M/I/C/S
1	Deal ID	string	C
2	Address	string	C
3	Offer Type	string	C
4	Offer Price	currency	C
5	Terms Summary	string	C
6	Cash Offer	currency	C
7	Sub2 Terms	string	C
8	Wrap Terms	string	C
9	Seller Carry Terms	string	C
10	Lease Option Terms	string	C
11	Hybrid Terms	string	C
12	Risk Notes	string	C
13	Sent Date	date	M
14	Response	string	M
15	Counter Terms	string	M
16	Status	string	M
17	Notes	string	M
18	Contract Sent	boolean	M
19	Buyer Assigned	string	M

Repair Estimator (25 columns)

Deal ID, Address, Year Built, Sqft, Property Type, Condition Notes, Roof, HVAC, Plumbing, Electrical, Foundation, Kitchen, Bathrooms, Flooring, Paint, Windows/Doors, Exterior, Landscaping, Other, Complexity Tier, Rehab Low, Rehab High, Rehab Mid, Risk Score, Notes.

Buyer Matching Engine (16 columns)

Deal ID, Address, Best Strategy, Asking Price, ARV, ZIP, Property Type, Match 1 Buyer, Match 1 Score, Match 2 Buyer, Match 2 Score, Match 3 Buyer, Match 3 Score, Suggested Dispo Action, Matched At, Dispo Status.

Lead Scoring & Risk (14 columns)

Deal ID, Address, Motivation Score, Equity Score, Market Score, Condition Score, Seller Response Score, Speed-to-Lead Score, SOM Impact, Total Lead Score, Risk Score, Combined Grade, Scoring Notes, Last Scored.

Post-Sale Tracker (19 columns)

Deal ID, Address, Strategy Used, Projected Sale Price, Actual Sale Price, Price Variance, Projected Rent, Actual Rent, Rent Variance, Projected Timeline Days, Actual Timeline Days, Timeline Variance, Projected Profit, Actual Profit, Profit Variance, Close Date, Notes, Lessons Learned, Tune Recommendations.

Staging Sheets (19 columns — shared schema)

Source Platform, Listing URL, Address Raw, City, State, ZIP, Price Raw, Beds Raw, Baths Raw, Sqft Raw, Lot Raw, Year Built Raw, Description Raw, Agent/Seller Name, Phone/Email, Scrape Timestamp, Scrape Job Link, Source Sheet Name, Processed.

Web & Ad Leads (17 columns)

Lead ID, Source, Campaign, Ad Set, Timestamp (Lead Arrival), Name, Email, Phone, Property Address, Property City, Property State, Property ZIP, Asking Price, Motivation, Timeline, Notes, Processed.

Import Hub (5 columns)

Source, Records Count, Last Import, Status, Action.

Settings (5 columns)

Setting Name, Value, Type (string/number/boolean/custom), Description, Last Updated.

System Log (4 columns)

Timestamp, Category, Message, Details.

Error Log (5 columns)

Timestamp, Module, Error Message, Stack Trace, Resolved.

Sync Log (6 columns)

Timestamp, CRM System, Action, Record ID, Status, Details.

Dashboard (5 columns)

Metric, Value, Change, Trend, Last Updated.

Control Center (5 columns)

Action, Status, Last Run, Next Scheduled, Notes.

7. Configuration Reference

All strategy engine defaults are in `CONFIG.STRATEGIES` and can be overridden at runtime via the Settings sheet.

Flip Engine Defaults

Setting	Default	Description
holdingCostMonthly	1% of purchase price	Monthly holding cost rate
agentFees	6% of ARV	Total buyer + seller agent commissions
closingCosts	3% of ARV	Total closing costs at sale
minProfitMargin	15%	Minimum acceptable profit margin
targetProfitMargin	25%	Target profit margin for full Flip Score

STR Engine Defaults

Setting	Default	Description
occupancyDefault	65%	Base occupancy rate
managementFee	20%	Property management / co-host fee
cleaningPerTurn	\$75	Cleaning cost per guest turnover
furnishingCost	\$8,000	Initial furniture cost
setupCost	\$2,000	Photos, supplies, initial setup
platformFee	3%	Airbnb/VRBO platform fee
seasonalityFactor	1.0	Base seasonality multiplier
regulationRiskDefault	0.5	Default STR regulation risk (0=none, 1=banned)

MTR Engine Defaults

Setting	Default	Description
avgStayLength	3 months	Average tenant stay duration
vacancyBetweenStays	0.5 weeks	Turnaround vacancy between tenants
utilitiesBundleMonthly	\$200	Monthly utilities included in MTR rent
furnitureAmortization	24 months	Furniture depreciation period
managementFee	12%	Property management fee

LTR Engine Defaults

Setting	Default	Description
vacancyRate	8%	Annual vacancy allowance
maintenanceReserve	10% of rent	Monthly maintenance reserve
capExReserve	5% of rent	Monthly CapEx reserve
propertyManagement	10% of rent	PM fee
rentGrowthAnnual	3%	Projected annual rent increase
targetDSCR	1.25	Minimum acceptable Debt Service Coverage Ratio

Creative Finance Defaults

Structure	Default Parameters
Sub-To	5% discount from asking; 24-month hold
Wrap	1–3% rate spread; minimum 3% down payment
Seller Carry	6% interest rate; 5-year term; 3-year balloon
Lease Option	3% option fee; 24-month term; 25% rent credit toward purchase

8. Scoring & Decision Logic

Deal Score Formula

```
Deal Score =  
  (Profit Score × 0.30) +  
  (Market Conditions Score × 0.25) +  
  (Motivation Score × 0.15) +  
  (Property Quality Score × 0.10) +  
  (Speed-to-Lead Score × 0.10) +  
  Strategy Bonus (additive) +  
  SOM Impact (additive)
```

Profit Score (30%): Based on MAO Final vs. Asking Price spread. Higher spread = higher score.

Market Conditions Score (25%): Composite of Sales Velocity Score, Exit Risk (inverse), and Market Heat Score.

Motivation Score (15%): Scored from Seller Type and keywords in Motivation Signals (distress keywords like "divorce", "foreclosure", "estate sale" boost score).

Property Quality Score (10%): Based on year built, property type, and Comp Confidence Score. Newer construction and confirmed comps score higher.

Speed-to-Lead Score (10%): OPTIMAL → 100, ACCEPTABLE → 75, SLOW → 50, BREACH → 0.

SOM Impact (additive): LOW saturation → +5, MOD → 0, HIGH → -5, SATURATED → -15.

Strategy Bonus (additive): +5 for HOT deals with strong strategy scores.

Risk Score Formula

```
Risk Score =
  (Repair Risk Score × 0.35) +
  (Exit Risk Score × 0.30) +
  (SOM Score × 0.20) +
  (100 - Comp Confidence Score) × 0.15
```

Verdict Thresholds

Verdict	Deal Score	Color	Default Action
HOT	≥ 80	Green (#4CAF50)	CALL NOW
SOLID	≥ 60	Blue (#2196F3)	MAKE OFFER
HOLD	≥ 40	Orange (#FF9800)	WATCH
PASS	< 40	Red (#F44336)	SKIP

Next Action Determination

`determineNextAction()` considers both Verdict and urgency signals:

- HOT + SLA OPTIMAL → CALL NOW
- HOT + SLA BREACH → CALL NOW (escalated)
- SOLID → MAKE OFFER
- HOLD + strong market velocity → RESEARCH
- HOLD + weak market → WATCH
- PASS → SKIP
- New Lead with no scores yet → RESEARCH
- Previously contacted → FOLLOW UP

9. CRM Integrations

SMS-iT

Type: Outbound sync
Trigger: Manual or pipeline step
Filter: HOT and SOLID deals only; skips already-synced records

Auth: Bearer token in Authorization header

Endpoint: `{crm_smsit_api_url}/contacts` (POST)

Payload fields: firstName, lastName, phone, email, address, city, state, zip, customFields (dealId, askingPrice, arv, verdict, bestStrategy, sellerMessage), tags

CompanyHub

Type: Outbound sync

Trigger: Manual or pipeline step

Filter: All non-PASS deals; skips already-synced

Auth: Bearer token

Endpoint: `{crm_companyhub_api_url}/deals` (POST)

Stage mapping: HOT → Qualified, SOLID → Interested, HOLD → Nurturing, PASS → Disqualified

Payload fields: name, type, stage, value, properties (address, city, state, zip, askingPrice, arv, dealScore, riskScore, bestStrategy, offerPrice), customFields (quantumDealId, verdict, nextAction)

Escalation tasks: SpeedToLead creates CRM tasks in CompanyHub for BREACH leads via `createCRMEscalationTask()` → POST to `{crm_companyhub_api_url}/tasks`.

OhMyLead

Type: Inbound webhook

Deployment: Google Apps Script Web App (`doPost()` handler)

Security: Optional webhook secret validated from `crm_ohmylead_secret` setting

Field mapping: Maps OhMyLead payload to Web & Ad Leads sheet with Lead Arrival Timestamp

Verification: `doGet()` returns status JSON for webhook verification calls

Retry Logic

All outbound CRM calls use `crmFetch()` :

- Up to 2 retries (3 total attempts)
- Exponential backoff: 1s after attempt 1, 2s after attempt 2
- No retry on 4xx client errors (except 429 rate limit)
- All calls logged to Sync Log sheet

10. Automation & Scheduling

Scheduled Triggers (Apps Script Time-Based)

Trigger	Handler	Frequency	Controlled by Setting
Nightly Refresh	<code>nightlyRefresh()</code>	Daily at 2 AM (timezone from Settings)	<code>auto_nightly_refresh</code>

Trigger	Handler	Frequency	Controlled by Setting
Dashboard Update	<code>refreshDashboard()</code>	Every 1 hour	<code>auto_dashboard_update</code>
Speed-to-Lead Check	<code>checkSpeedToLeadSLA()</code>	Every 5 minutes	<code>auto_stl_check</code>
CRM Sync	<code>syncToCRMIfEnabled()</code>	Configurable (part of nightly or manual)	CRM enabled flags
onOpen	<code>onOpen()</code>	On spreadsheet open	Always active

Trigger Management

- `setupAutomationTriggers()` — Deletes all existing triggers and recreates based on current Settings toggles. Called via "Settings & Admin" menu or `toggleAutomation()`.
- `createTriggers()` — Simpler version in Code.gs; creates 3 core triggers unconditionally.
- `removeAllTriggers()` — Deletes all triggers (useful for debugging).

Concurrent Run Prevention

`runFullPipelineSafe()` wraps `runFullPipeline()` in `LockService.getScriptLock()` with a 5-second wait. If the lock is not acquired (another run is in progress), it logs the skip and returns `{skipped: true}`.

Nightly Refresh Sequence

```

nightlyRefresh()
├─ importFromStaging()
├─ runAnalyzeAndScore()
│   ├─ computeAllScores()
│   └─ runAllStrategyEngines()
└─ runMultiExitComparison()
├─ refreshDashboard()
└─ archiveOldLogs() (removes logs older than 30 days)

```

11. HTML Interfaces

Six HTML dialog interfaces are served from the `html/` subdirectory of `src/`. Each is opened as a modal dialog via Google Apps Script's `HtmlService`.

Dialog	File	Size	Purpose
Setup Wizard	<code>setup-wizard.html</code>	900×700	Initial system configuration walkthrough
Control Center	<code>control-center.html</code>	1200×800	Live dashboard, trigger management, CRM status, recent logs
Deal Analyzer	<code>deal-analyzer.html</code>	1100×750	Per-deal deep-dive analysis viewer
Offer Generator	<code>offer-generator.html</code>	1000×700	Generate and review offer packs per deal

Dialog	File	Size	Purpose
Buyer Matcher	buyer-matcher.html	1000×700	View buyer matches; add/manage buyers
Help / SOP	help-sop.html	900×650	Standard operating procedures and help documentation

Data flow for HTML UIs: All dialogs call `google.script.run` to invoke Apps Script functions (e.g., `getControlCenterData()`, `getActiveBuyers()`, `getOfferPackForDeal()`). Responses are returned asynchronously via callback functions.

12. Data Identifiers & Deduplication

Primary Keys

Identifier	Sheet	Format	Example
Deal ID	Master Database (col 1) + all child sheets	<code>Q</code> + base-36 timestamp + 4-char random	<code>QLXF4A2BKPQR</code>
Buyer ID	Buyer Database (col 1)	<code>B</code> + base-36 timestamp	<code>B1F4A2BKPQ</code>
Lead ID	Web & Ad Leads (col 1)	<code>WL</code> + base-36 timestamp	<code>WL1F4A2BKPQ</code>

Deduplication Keys

Key	Usage	Format
Address + ZIP (primary)	Main dedup: catches same property from multiple sources	Lowercase, alphanumeric only, joined with <code> </code>
Listing URL (secondary)	Catches same listing URL across imports	Lowercased, trimmed

Dedup Algorithm

1. Group all Master DB rows by normalized `address|zip` key.
2. Any group with > 1 row contains duplicates.
3. Secondary pass: group by normalized Listing URL.
4. For each duplicate group:
 - Calculate data quality score for each row (count of non-empty important fields).
 - Keep the highest-scoring row.
 - Merge non-empty field values from lower-scoring rows into the keeper (fills gaps).
 - Delete lower-scoring duplicate rows.
5. Fuzzy matching via Levenshtein distance (threshold: edit distance $\leq$ 10% of max string length) handles minor address variations.

13. Key Algorithms

Deal ID Generation

```
'Q' + Date.now().toString(36).toUpperCase() + randomString(4)
```

Base-36 timestamp ensures rough chronological ordering. 4-character random suffix prevents collisions.

ZIP Dedup Normalization

```
function normalizeForDedup(address, zip) {  
  const normAddr = address.toString().toLowerCase().replace(/^[a-z0-9]/g, '');  
  const normZip = zip.toString().replace(/^\D/g, '').substring(0, 5);  
  return normAddr + '|' + normZip;  
}
```

Flip MAO (70% Rule)

```
MAO_Flip = (ARV × 0.70) - Rehab_Mid
```

STR Monthly Net

```
Gross = ADR × 30 × occupancy × seasonality × (1 - regulationRisk × 0.3)  
CleaningCosts = cleaningPerTurn × (occupancy × 30 / avgStayDays)  
Net = Gross - CleaningCosts - (Gross × mgmtFee) - (Gross × platformFee)
```

LTR DSCR

```
DSCR = NOI_Annual / AnnualMortgagePayment
```

Target ≥ 1.25 . LTR Score penalizes when $DSCR < 1.0$.

Standard Mortgage Amortization (used in SellerCarry, LTR, and Creative Engine)

```
function calculateMortgagePayment(principal, annualRate, termYears) {  
  const monthlyRate = annualRate / 12;  
  const n = termYears * 12;  
  return principal * (monthlyRate * Math.pow(1 + monthlyRate, n)) / (Math.pow(1 + monthlyRate, n) - 1);  
}
```

Buyer Match Score

```
Score = ZIP_match (0/20/40) + Strategy_match (0/15/25) + Budget_match (0/10/20) + PropType_match (0/10) + ...
Max possible = 100
```

SLA Clock

```
ElapsedMinutes = (Now - Lead_Arrival_Timestamp) / 60000
SLA_Clock stops when Last_Contacted_At is recorded
```

Exit Risk Score

```
ExitRisk = (normalizedDOM × 0.40) + (repairRisk × 0.30) + (somScore × 0.20) + ((100 - compConfidence) × 0.10)
```

14. Settings Sheet Reference

The Settings sheet is initialized by `initializeSettingsSheet()` with these default rows:

General

Setting Name	Default Value	Description
system_name	Quantum Real Estate Analyzer 2.0	Display name
timezone	America/New_York	Trigger timezone
currency	USD	Currency format

Flip Strategy

Setting Name	Default
flip_holding_cost_monthly	0.01
flip_agent_fees	0.06
flip_closing_costs	0.03
flip_min_profit_margin	0.15
flip_target_profit_margin	0.25

STR Strategy

Setting Name	Default
str_default_occupancy	0.65
str_management_fee	0.20
str_cleaning_per_turn	75
str_furnishing_cost	8000
str_platform_fee	0.03

MTR Strategy

Setting Name	Default
mtr_avg_stay_length	3
mtr_vacancy_between_stays	0.5
mtr_utilities_bundle	200
mtr_management_fee	0.12

LTR Strategy

Setting Name	Default
ltr_vacancy_rate	0.08
ltr_maintenance_reserve	0.10
ltr_capex_reserve	0.05
ltr_property_management	0.10
ltr_target_dscr	1.25

Creative Finance

Setting Name	Default
creative_sub2_discount	0.05
creative_wrap_spread_min	0.01
creative_wrap_spread_max	0.03
creative_seller_carry_rate	0.06
creative_lease_option_fee	0.03

Speed-to-Lead

Setting Name	Default
stl_tier1_minutes	5
stl_tier2_minutes	15
stl_tier3_minutes	60
stl_breach_penalty	-25

CRM

Setting Name	Default
crm_smsit_enabled	false
crm_smsit_api_url	(empty)
crm_smsit_api_key	(empty)
crm_companyhub_enabled	false
crm_companyhub_api_url	(empty)
crm_companyhub_api_key	(empty)
crm_ohmylead_enabled	false
crm_ohmylead_webhook	(empty)
crm_ohmylead_secret	(empty)

AI / Messaging

Setting Name	Default
ai_openai_enabled	false
ai_openai_api_key	(empty)
ai_model	gpt-4o-mini
ai_max_tokens	500

Market Data API

Setting Name	Default
market_data_api_url	(empty)
market_data_api_key	(empty)

Automation Toggles

Setting Name	Default
auto_nightly_refresh	true
auto_dashboard_update	true
auto_stl_check	true

End of System Documentation. This document covers all 17 source modules, 27 sheets, all column schemas, configuration defaults, scoring algorithms, pipeline execution order, CRM integrations, and automation scheduling for the Quantum Real Estate Analyzer 2.0.